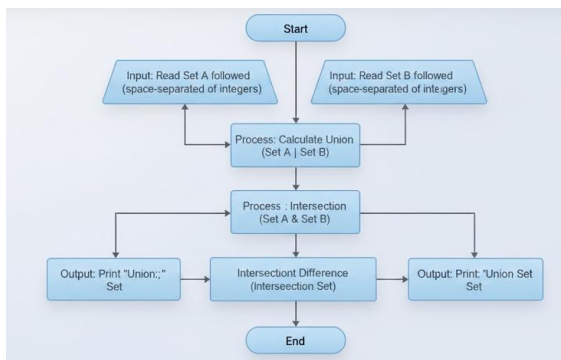


4.1.1 Set operation

Algorithm: Set Operations Program

1. **Start.**
2. **Input Phase:**
 - Read a line of input for **Set A**.
 - Split the string by spaces to get individual numeric strings.
 - Convert each string to an integer and store them in a **Set** data structure (this automatically handles duplicates).
 - Repeat these steps for **Set B**.
3. **Processing Phase:**
 - **Union:** Create a new set containing all unique elements from both Set A and Set B.
 - **Intersection:** Create a new set containing only the elements that exist in both Set A and Set B.
 - **Difference:** Create a new set containing elements that are in Set A but **not** in Set B.
4. **Output Phase:**
 - Print "Union: " followed by the Union set.
 - Print "Intersection: " followed by the Intersection set.
 - Print "Difference: " followed by the Difference set.
5. **End.**

Flowchart:



The screenshot shows the CodeTANTRA IDE interface. On the left, the file explorer shows '4.1.1. Set Operations'. The main editor displays the following Python code:

```
1 set_a = set(map(int, input("Set A: ").split()))
2 set_b = set(map(int, input("Set B: ").split()))
3 union_set = set_a.union(set_b)
4 intersection_set = set_a.intersection(set_b)
5 difference_set = set_a.difference(set_b)
6 print("Union:", union_set)
7 print("Intersection:", intersection_set)
8 print("Difference:", difference_set)
```

Below the code, the test results are shown. The program passed 2 out of 2 shown test cases and 0 out of 2 hidden test cases. The test cases are as follows:

Test Case	Expected Output	Actual Output
Test case 1	Set A: 1 2 3 4 5 6 Set B: 3 4 5 6 7 8 Union: {1, 2, 3, 4, 5, 6, 7, 8} Intersection: {3, 4, 5, 6} Difference: {1, 2}	Set A: 1 2 3 4 5 6 Set B: 3 4 5 6 7 8 Union: {1, 2, 3, 4, 5, 6, 7, 8} Intersection: {3, 4, 5, 6} Difference: {1, 2}